

Corso di Laurea in Informatica A.A. 2024-2025

Laboratorio di Sistemi Operativi

Alessandra Rossi

Funzione Main

Un programma C inizia la sua esecuzione obbligatoriamente con la chiamata alla funzione **main**. Lo standard C stabilisce che la funzione main può **non** avere argomenti o prendere i **due** argomenti **argc** e **argv[]**:

```
int main (void);  
int main (int argc, char *argv[ ]);
```

argc rappresenta il numero di argomenti passati in input (incluso il nome del programma, definito dal nome del file contenente la chiamata a main);

argv è un vettore di puntatori a carattere e rappresenta la **lista di argomenti** (opzioni e parametri) del programma, con **argv[0]** che rappresenta il nome del *programma*

Lista di Ambiente

Ad ogni **programma** viene passata, oltre alla lista degli argomenti di input, una **lista d'ambiente**. Essa serve alle applicazioni per definire il contesto in cui operano (directory home, tipo di terminale, nome utente, etc.)

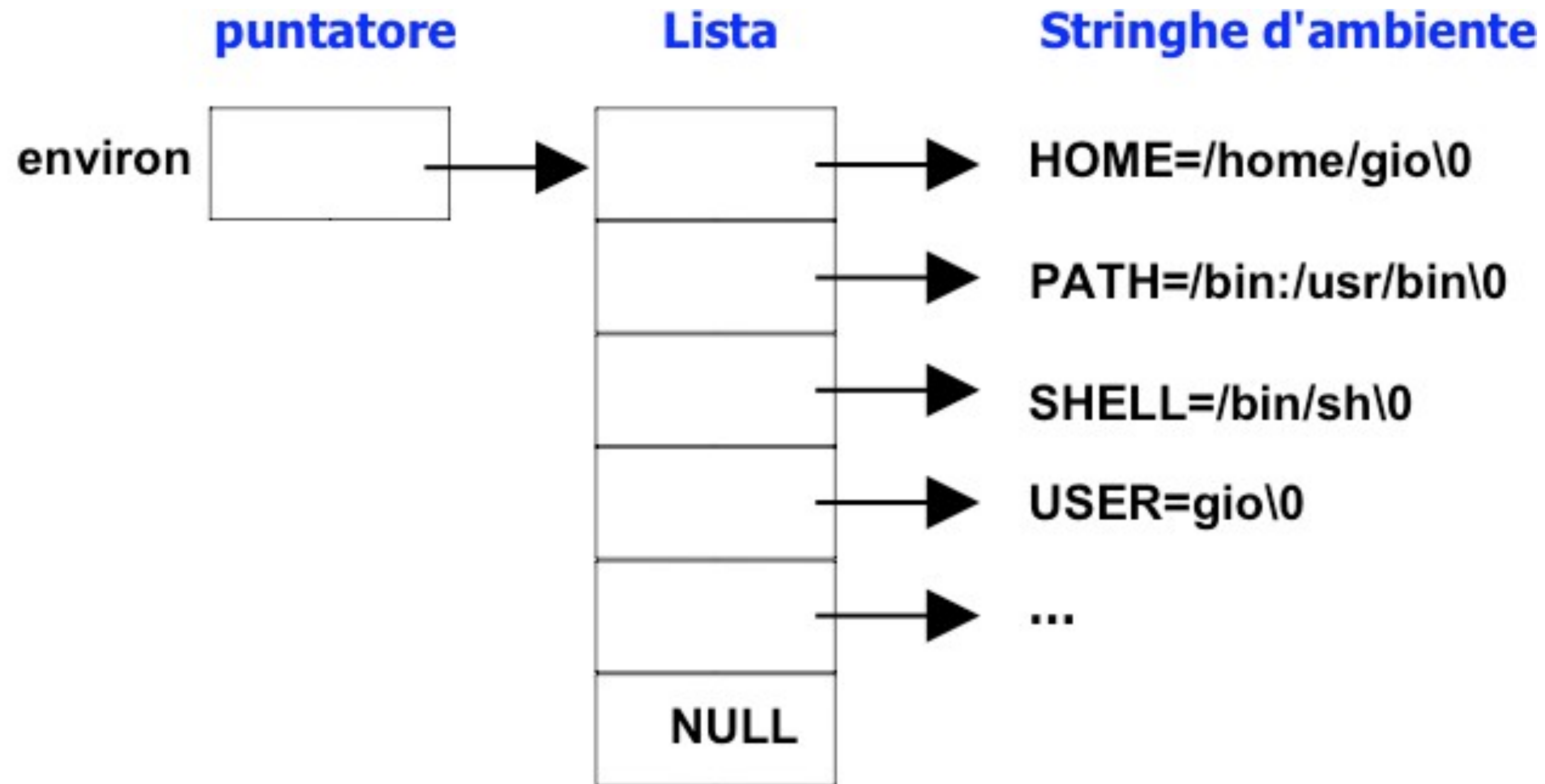
Analogamente alla lista degli argomenti, la **lista d'ambiente** è un array di puntatori a carattere, in cui l'ultimo puntatore punta a NULL.

Ciascun puntatore della lista contiene l'indirizzo di una stringa del tipo **nome = valore**, detta **stringa d'ambiente**.

L'indirizzo dell'array di puntatori è contenuto nella variabile globale **environ**:

```
extern char **environ;
```

Lista di Ambiente



Variabili di Ambiente

Le **variabili d'ambiente** sono definite e modificate operando sulle **stringhe d'ambiente**, grazie ad opportune funzioni. Le due più importanti sono:

```
# include <stdlib.h>
```

```
char *getenv(const char *name);
```

```
int putenv(char *string);
```

ritorna zero se OK, non zero se ENOMEM

getenv restituisce il puntatore alla stringa **valore**, associata al nome della variabile d'ambiente **nome** nella stringa d'ambiente **nome=valore**. Se **nome** non esiste restituisce il puntatore nullo.

putenv aggiunge alla lista d'ambiente la stringa **string** della forma **nome=valore**. Se **nome** esiste ne aggiorna il valore.

Layout in memoria di un Prog.

Questo segmento dell'area di memoria riservata ad un programma serve per la memorizzazione degli argomenti della linea di comando e delle variabili d'ambiente.

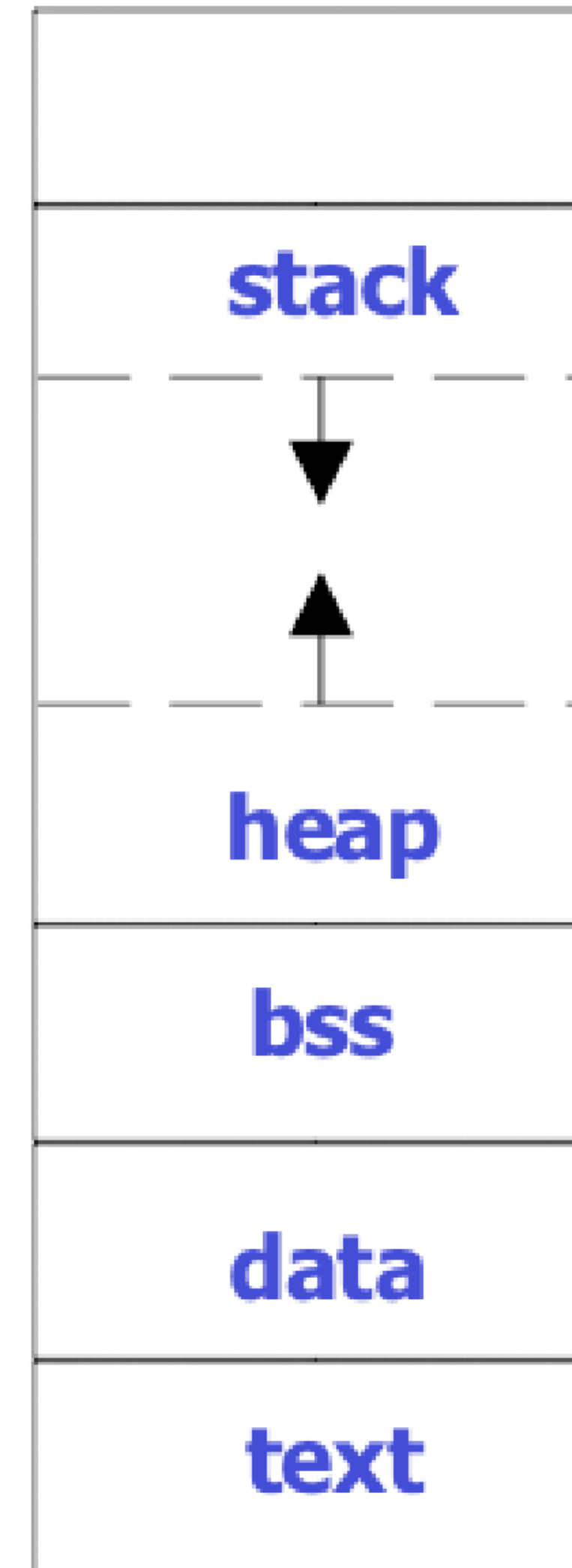
Lo **stack** serve a memorizzare le variabili locali e le informazioni relative alle chiamate di funzioni. Lo *stack* è implementato come una coda *LIFO* (*Last In, First Out*), per permettere l'uso annidato e ricorsivo delle funzioni.

Lo **heap** è dove viene effettuata l'allocazione dinamica di memoria (funzioni **malloc**, **calloc**, **realloc**, **free**,...). Anche quest'area è implementata come una coda LIFO, ed è tipicamente gestita a basso livello dalla syscall **sbrk**.

Le variabili globali non inizializzate, analogamente ai puntatori a variabili globali, vengono memorizzati nel segmento **bss**, detto anche *uninitialized data segment*. I dati in tale segmento sono inizializzati dal kernel come valori numerici pari a zero o puntatori nulli prima che il programma vada in esecuzione.

L'area **data** contiene le variabili globali ed inizializzate del programma.

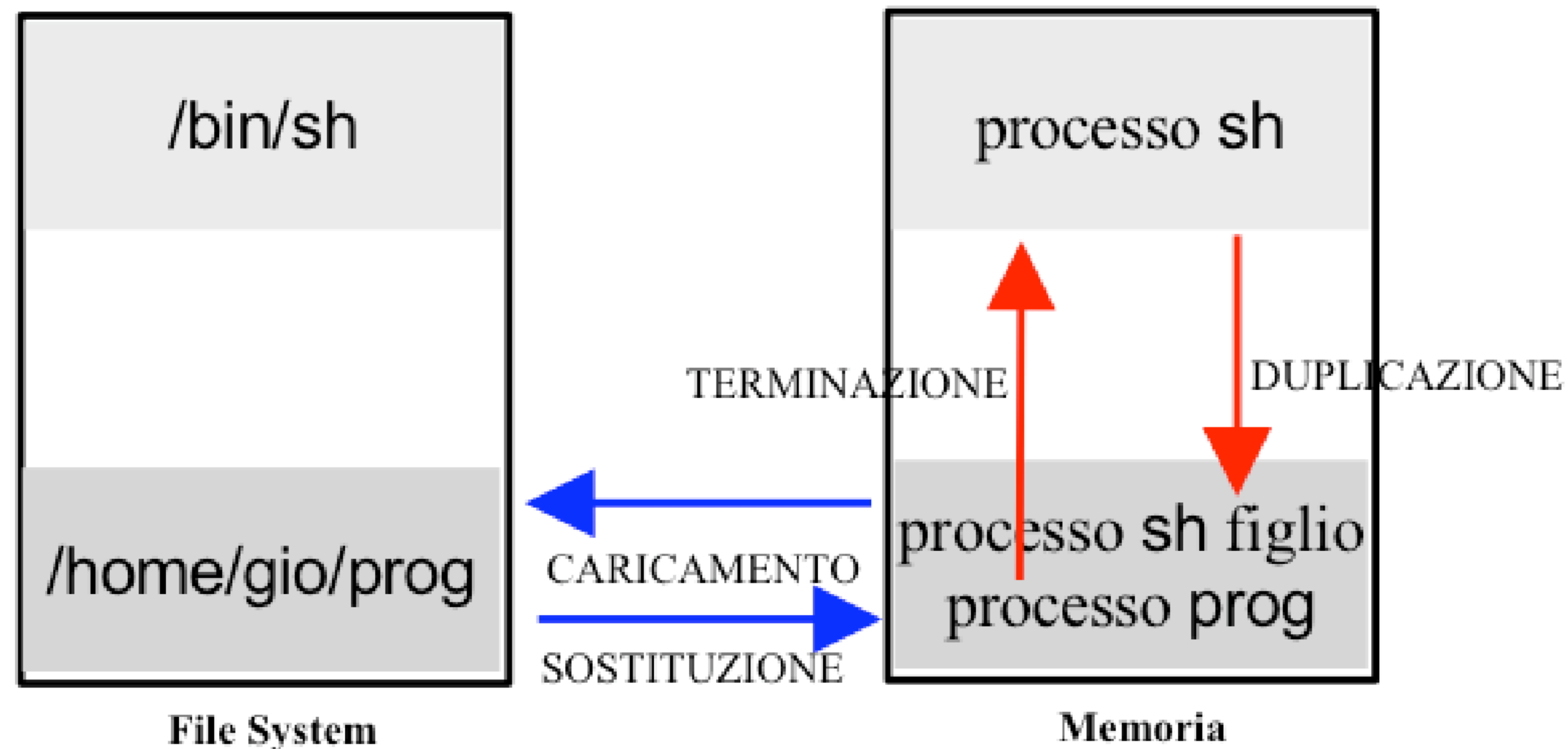
L'area **text** contiene le istruzioni macchina relative al programma. Quest'area è in sola lettura, perchè condivisa tra tutti i processi che eseguono uno stesso codice binario.



Esecuzione di un Prog.

Qual'è il meccanismo per l'esecuzione di un nuovo programma in UNIX?

```
$ prog <invio>  
<eventuale output di prog>  
$
```



Esecuzione del Prog.

- Quando il prog. viene eseguito (una funzione exec), una routine start-up prende i command-line args e l'env prima del lancio della funzione main
- La fine dell'esecuzione avviene in diversi modi:
return, exit, _exit, pthread_exit, abort, segnale, etc.

Controllo processi

Per il controllo dei processi occorrono primitive per gestire:

- Creazione di Processi
- Esecuzione di Programmi
- Terminazione di Processi
- Identificazione e proprietà dei processi

Controllo di Processi

Il controllo dei processi in UNIX si esplica mediante:

- L'identificazione dei processi:
 - Identificatore di processo (**pid**);
 - Identificatore di gruppo-processi (**pgid**);
 - Identificatori di utente (famiglia **uid**);
 - Identificatori di gruppo-utenti (famiglia **gid**);
- L'impiego di funzioni (**primitive di controllo di processo**) per:
 - Duplicare** un processo esistente (**fork**, **vfork**) ;
 - Caricare** un nuovo programma (famiglia **exec**);
 - Attendere** la terminazione di un processo (**wait**, **waitpid**);
 - Terminare** un processo (**exit**, **_exit**).

Identificazione di Processi

I processi in UNIX si dividono in processi **di sistema** e di **utente**;

Ogni processo è identificato da un **numero non negativo (PID)**; unico PID, però riciclato quando termina;
PID=0 scheduler, PID=1 init, PID = 2 pagedaemon

Ogni processo **utente** eredita una serie di altri identificativi:

- Il **gruppo di processi** cui esso appartiene (**PGID**);
- Lo UID e il GID dell'**utente** che lo ha mandato in esecuzione (**RUID** e **RGID**);
- Lo UID e i GID (gruppi primario e supplementari) di utente **con i quali esso ha accesso ai file** (**EUID, EGID, suppl. EGID**);

Identificazione di Processi

- Ogni processo ha un pid ed un Parent's pid (ppid)
 - i processi formano un albero
- “init” e' la radice dell'albero
 - viene lanciato direttamente dal kernel
 - non ha padre
 - ha pid=1
- quando un processo termina, i suoi figli diventano figli di “init”

Identificazione di Processi

Le seguenti funzioni restituiscono gli **identificativi** e le **credenziali di processo**:

```
#include <sys/types.h>
#include <unistd.h>

pid_t getpid(void);    identificativo del processo    PID
pid_t getppid(void);   identificativo del genitore    PPID
uid_t getuid(void);     credenziale utente reale    RUID
uid_t geteuid(void);    credenziale utente effettivo    EUID
gid_t getgid(void);     credenziale gruppo reale    RGID
gid_t getegid(void);    credenziale gruppo effettivo    EGID
```

Creazioni di Processi

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
pid_t fork(void);
```

```
pid_t vfork(void);
```

(vfork da usare con exec, lavora nello spazio del genitore e aspetta il figlio)

ritornano il PID se OK, -1 se EAGAIN, ENOMEM o EPERM

L'**unico modo** per istruire il kernel a **creare** un nuovo **processo** è di chiamare la funzione **fork** (**vfork**) da un processo esistente. Il **processo** creato viene detto **figlio**. Il processo che chiama **fork** (**vfork**) viene detto **genitore**.

Ogni chiamata a **fork** (**vfork**) ha **due** ritorni:

- al **genitore** viene restituito l'**identificativo del figlio**;
- al **figlio** viene restituito l'**identificativo 0**.

Creazione di Processo

Esempio tipico:

```
pid_t pid;  
  
if ( (pid=fork()) < 0 )  
    perror("fork"), exit(1);  
else if (pid != 0) {  
    // codice del padre  
} else {  
    // codice del figlio  
}
```

Padri e Figli

- Il figlio procede indipendentemente dal padre
- **memoria**: il figlio ottiene una copia nuova della memoria del padre (variabili globali e locali)
- **file aperti**: i descrittori vengono copiati come con dup; i processi condividono l'offset!
- **segnali**: per ogni segnale, il figlio continua ad avere la stessa reazione del padre

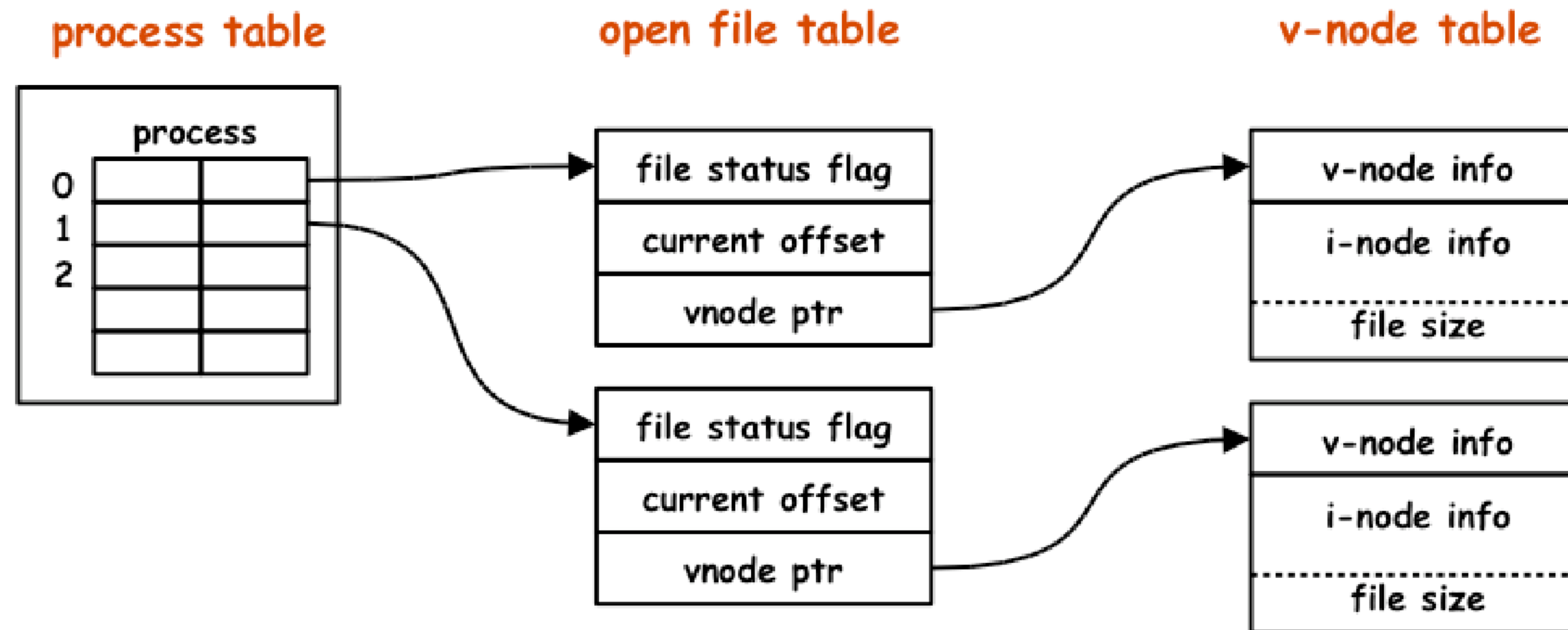
Creazione di Processi-fork

Ad una chiamata **fork** il **kernel** esegue le operazioni seguenti:

- Alloca uno spazio nella tabella dei processi per il figlio;
- Assegna un PID al figlio, unico nel sistema;
- Fa una copia dell'immagine del genitore, ad eccezione dei segmenti di memoria **condivisi**;
- Incrementa i contatori dei file del genitore, per registrare che anche il figlio possiede tali file;
- Assegna al processo figlio lo stato READY;
- Restituisce il PID del figlio al genitore e il PID 0 al figlio;
- A seconda della **routine di allocazione**, può:
 - rimanere nel genitore;
 - trasferire il controllo al figlio;
 - trasferire il controllo ad un altro processo.

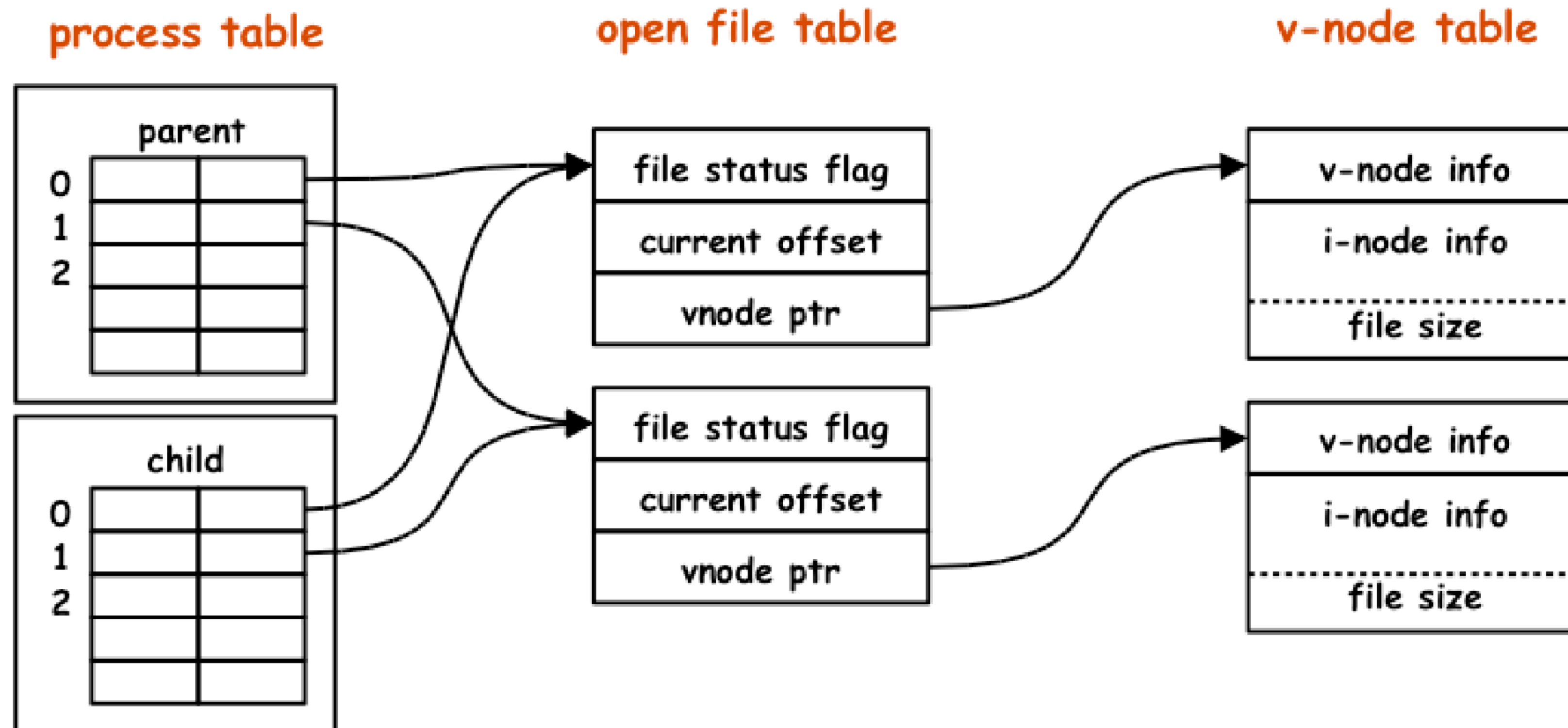
Fork e File

- Una caratteristica della chiamata `fork` è che tutti i descrittori che sono aperti nel processo parent sono duplicati nel processo child
- Prima della `fork`:



Fork e File

- Dopo la `fork`:



Fork e File

- E' importante notare che padre e figlio condividono lo stesso *file offset*
- Consideriamo il seguente caso:
 - un processo esegue `fork` e poi attende che il processo figlio termini (system call `wait`)
 - supponiamo che lo `stdout` sia rediretto ad un file, e che entrambi i processi scrivano su `stdout`
 - se padre e figlio non condividessero lo stesso offset, avremmo un problema:
 - il figlio scrive su `stdout` e aggiorna il proprio current offset
 - il padre sovrascrive `stdout` e aggiorna il proprio current offset

Esercizio

- Scrivere un programma C che apre un file, effettua una **fork** e scrive messaggi diversi sul file a seconda che sia padre o figlio. Come si alternano i messaggi nel file?
- Nel programma prima descritto spostare la open dopo la **fork** e verificare se il contenuto del file è cambiato rispetto al programma precedente e spiegarne il perchè.

Esempio

```
int main(void) {  
    int i;  
  
    for (i=0; i<2 ;i++)  
        if (fork()>0) {  
            printf("Padre! %d\n", i);  
        } else {  
            printf("Figlio! %d\n", i);  
        }  
  
    sleep(10);  
    return 0;  
}
```

- Qual è l'output di questo programma?
- Quanti processi vengono creati?
- Di chi è figlio ciascun processo creato?

Esempio

```
$/a.out  
padre 0  
padre 1  
figlio! 0  
padre 1  
figlio! 1  
figlio! 1
```

```
padre 0  
padre1 figlio! 1
```

```
figlio! 0  
padre1 figlio! 1
```

Esempio 2

```
/* fork1.c: fork e le immagini in memoria di genitore e figlio */
#include <sys/types.h> /* per il tipo pid_t */
#include <unistd.h>    /* per le funzioni fork e sleep */
#include <stdlib.h>     /* per la funzione exit */
#include <stdio.h>      /* per la funzione printf */

int glob_ini=1; /* var. globale inizializzata --> data segment */
int glob;       /* var. globale non inizializzata --> bss segment */

int main(void)
{
    /* variabili locali --> stack */
    int local=10;
    pid_t pid;
    printf("Prima di chiamare fork\n"); /* Output con buffer */
    /* chiamata a fork */
    if ( ( pid=fork( ) ) < 0 ) /* Si e' verificato un errore di fork */
    {
        printf(" errore di fork");
        return 1;
    }
}
```

Esempio 2 (cont.)

```
else if ( pid == 0 )  
{  
    glob_ini++;  
    glob=5;  
    local++;  
}
```

```
else /* Il genitore aspetta nullafacente per 2 secondi */  
    sleep(2);
```

```
/* entrambi i processi scrivono sullo std output */  
printf("pid= %d, glob_ini= %d, glob= %d, local= %d\n", getpid( ),  
glob_ini, glob, local);  
return 0;  
}
```

Eseguendo fork1, si ottiene:

Esempio 2 (cont.)

```
gio$ ./fork1
Prima di chiamare fork
pid= 7183, glob_ini= 2, glob= 5, local= 11
pid= 7182, glob_ini= 1, glob= 0, local= 10
gio$ ./fork1 >fork1.out
gio$ cat fork1.out
Prima di chiamare fork
pid= 7189, glob_ini= 2, glob= 5, local= 11
Prima di chiamare fork
pid= 7188, glob_ini= 1, glob= 0, local= 10
```

} figlio

} genitore

Funzione vfork()

```
#include <unistd.h> pid_t vfork(void);
```

Crea nuovo processo come fork, ma non copia spazio indirizzamento, eseguito nello spazio del genitore finche' figlio non esegue exec o exit.

Finche' non eseguito exec o exit, figlio eseguito per primo vfork utilizzato con exec per l'esecuzione di un programma

Esempio

```
#include <systype.h>
#include "ourhdir.h" /* tutti header necessari*/

int glob = 6;
int main(void)
{ int var;
  pid_t pid;
  var =88;
  printf("before vfork\n");
  if ((pid =vfork())) perror("vfork"), exit(0);
  else if (pid==0) {
    glob++; var++; /* cambia variabili del padre */
    _exit(0);}
  printf("pid = %d glob = %d var = %d",getpid(), glob,var);
  exit(0);
}
```

Esempio

Eseguendo:

```
$/a.out
```

```
before vfork
```

```
pid = 2624, glob = 7, var = 89
```

Cioe', l'incremento della var del figlio cambia il
valore nel genitore.

Race Condition

Una **race condition** (condizione di tempificazione) si verifica quando più processi elaborano dati **condivisi** e l'effetto dell'insieme di siffatte elaborazioni **dipende dall'ordine** in cui i processi sono eseguiti.

L'ordine in cui vengono elaborate le istruzioni per un genitore ed un figlio dopo un **fork** (**vfork**) in generale dipende dallo scheduler e dal carico del sistema.

Una chiamata **fork** (**vfork**) può causare una race condition, se le istruzioni relative al genitore ed al figlio **operano su dati condivisi**;

Il rilevamento **in base a run** di una race condition del tipo suddetto può essere molto difficile.

Esempio 8

```
/* racecond.c: Fornisce un esempio di condizione di
tempificazione */
#include <sys/types.h> /* per il tipo pid_t */
#include <unistd.h>     /* per fork ed _exit */
#include <stdio.h>      /* per printf e putc */
#include <stdlib.h>     /* per exit */

static void char_char(char *); /* dich. di char_char */

void main(void)
{
    pid_t  pid;

    if ( (pid = fork()) < 0)
        printf("errore di fork"), exit(1);
    else if (pid == 0) {
        char_char("io sono il figlio, e intendo scrivere prima del
padre\n");
        _exit(0);
    }
}
```



Esempio 8 (cont.)

```
else {
    char _char("io sono il padre, e non ci sto\n");
    exit(0);
}
/* definizione di char_char */

static void char_char(char *str)
{
    char *ptr;
    int  c;

    setbuf(stdout, NULL); /* disattiva il buffer per stdout */
    for (ptr = str; c= *ptr++; )
        putc(c, stdout);
}
```

Eseguendo l'esempio, si ottiene:

```
gio$ io sono il padre, ei non oc i sosnto  
o il figlio, e intendo scrivere prima delpgio$ adre
```

Osservazione: In questo esempio, il padre termina prima che il figlio abbia finito di scrivere sullo `stdout`, come si evince dalla comparsa prematura del prompt. Il prompt alla terminazione del figlio non appare perchè quest'ultimo per uscire richiama `_exit`, che non restituisce il controllo al chiamante (la shell).

Esercizio

- Programma che genera due processi, il primo scrive una stringa su di un file, il secondo la legge.

Esercizio

```
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

#define BUFSIZE 4096 /* max. num. di caratteri in buf */

int main(int argc, char* argv[]){
    pid_t pid;    int status;    int fd;    int n;
    char buffer[BUFSIZE];    /* controlla i parametri input */
    if (argc != 3)
        printf("usage: trasfile <file> <stringa>"), exit(1);
    /* genera il processo figlio */
    if ( (pid = fork()) < 0)
        perror("fork"), exit(1);
```

```

/* errore di fork */
else if (pid == 0) { /* figlio */
    /* apre il file da scrivere */
    if ((fd = open(argv[1], O_WRONLY|O_CREAT|O_TRUNC, S_IRWXU)) < 0)
        perror("create file error"), exit(1);

    /* scrive la stringa */
    if ((n = write(fd, argv[2], strlen(argv[2]))) < 0)
        printf("write error"), exit(1);
    /* chiude il file */
    close(fd);
} else { /* padre */
    /* attende la terminazione del figlio */ if
    ( (pid = waitpid(pid, &status, 0)) < 0)
        perror("waitpid error"), exit(1);
    /* apre il file */
    if ((fd = open(argv[1], O_RDONLY, S_IRWXU)) < 0)
        perror("open file error"), exit(1);
    /* legge la stringa */
    if ((n = read(fd, buffer, BUFSIZE)) < 0)
        printf("read error"), exit(1);
    /* stampa la stringa */
    printf("trovato: %s", buffer);
    /* chiude il file */
    close(fd);
}
exit(0);
}

```

Esecuzione di Programmi

L'esecuzione di un **nuovo programma** in UNIX si ottiene con la chiamata ad una delle funzioni **exec**;

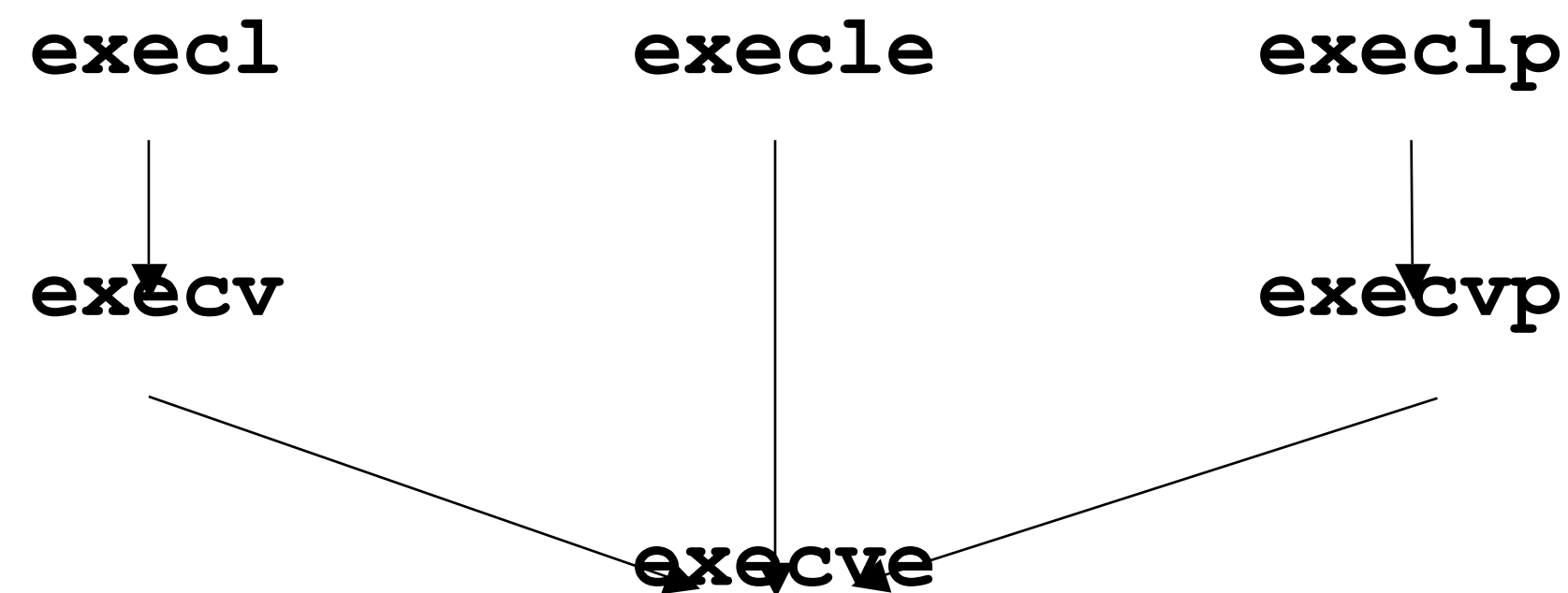
Le funzioni **exec** *non* generano un nuovo processo, ma **modificano il layout in memoria** del processo chiamante per l'esecuzione del nuovo programma;

Quando un processo chiama una delle funzioni **exec**, le aree di memoria **text**, **data**, **heap** e **stack** relative al processo corrente vengono sostituite in base al nuovo programma;

Il nuovo programma incomincia la propria esecuzione a partire dalla funzione **main**.

La famiglia di system call `exec`

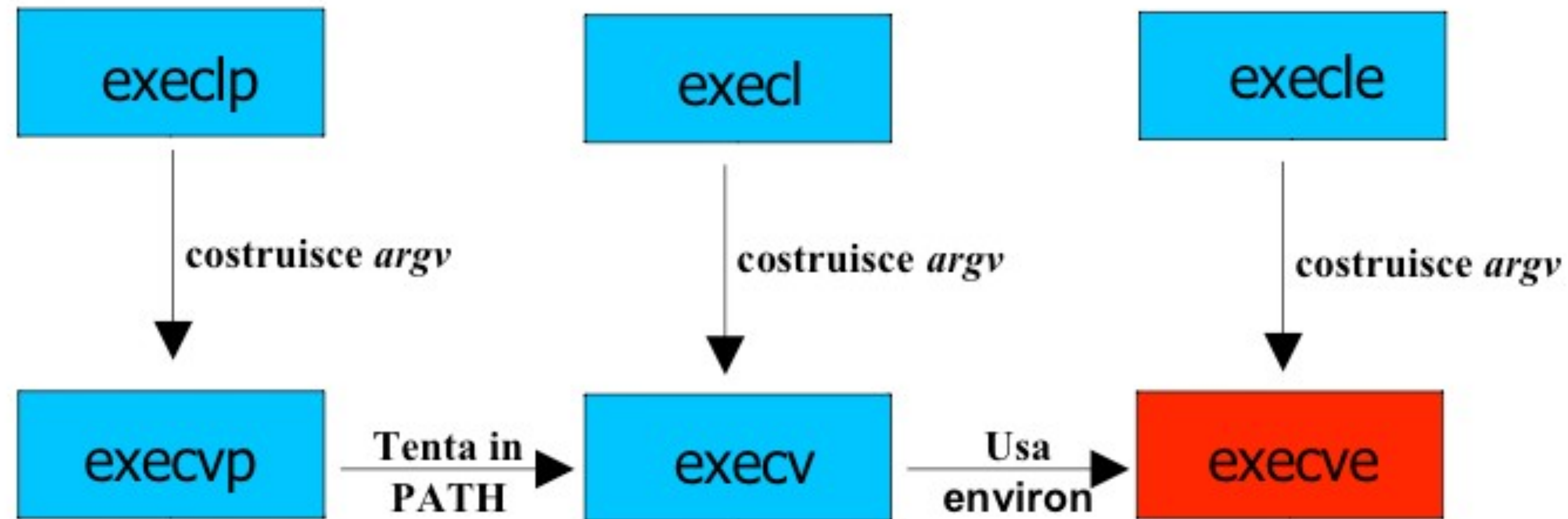
- Se `fork` fosse l'unica primitiva per creare nuovi processi, la programmazione in ambiente Unix sarebbe ostica, dato che si potrebbero creare soltanto copie dello stesso processo.
- La famiglia di primitive `exec` può essere utilizzata per superare tale limite in quanto le varie system call `exec` permettono di iniziare l'esecuzione di un altro programma sovrascrivendo la memoria del processo chiamante.



- In realtà tutte le funzioni chiamano in ultima analisi **`execlp`** che è l'unica vera system call della famiglia. Le differenze tra le varianti stanno nel modo in cui vengono passati i parametri.

La famiglia di system call `exec`

In molte implementazioni UNIX, solo `execve` è una *chiamata di sistema*, le altre cinque sono semplicemente funzioni di libreria *che invocano `execve`*.



Suffisso e parametri di `exec`

Funzione	pathname	filename	arg list	argv[]	environ	envp[]
<code>execl</code>	●		●		●	
<code>execlp</code>		●	●		●	
<code>execle</code>	●		●			●
<code>execv</code>	●			●	●	
<code>execvp</code>		●		●	●	
<code>execve</code>	●			●		●
lettere del suffisso		p	l	v		e

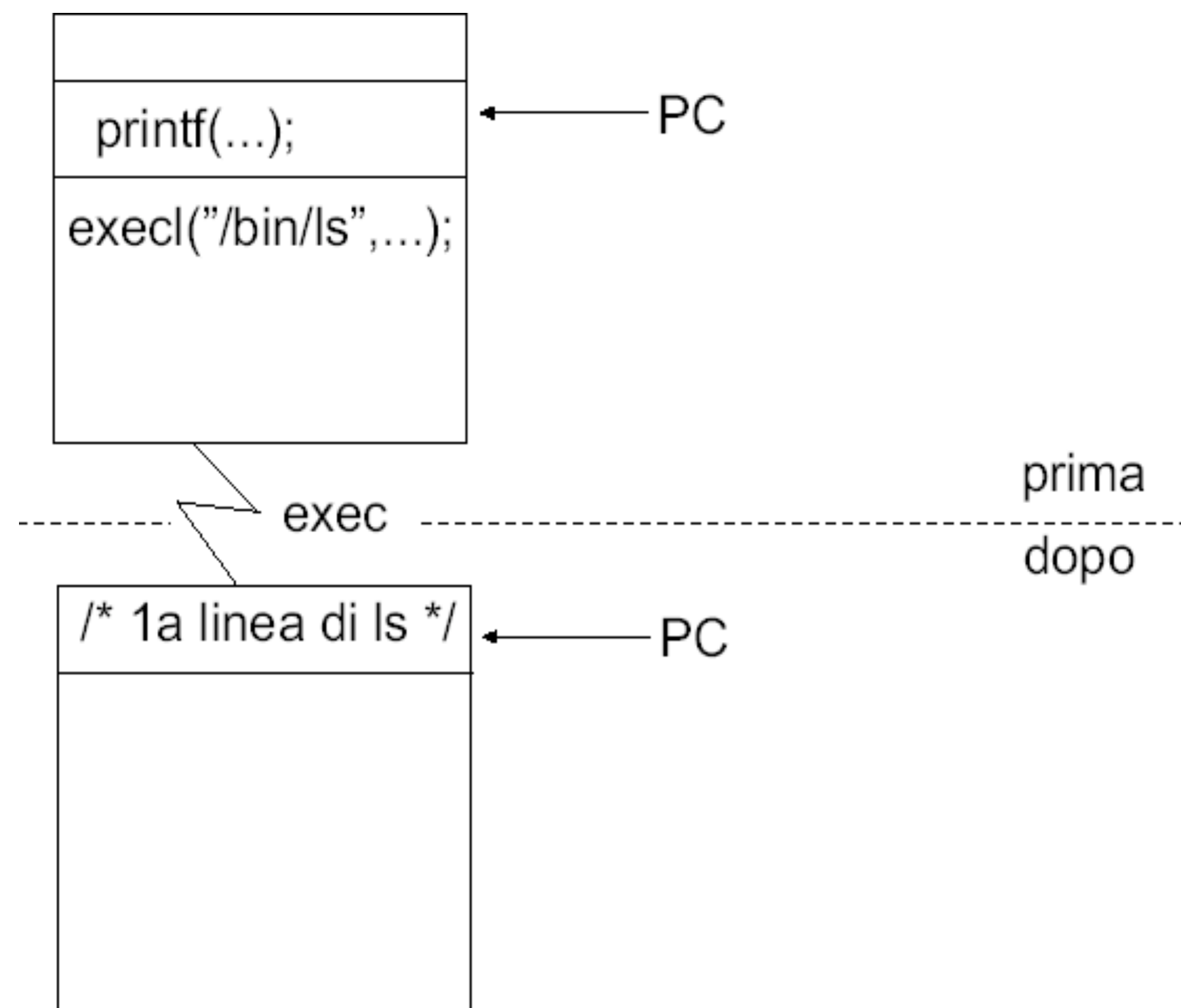
Funzione	Caratteristica
p	Non serve specificare il path.
v	Gli argomenti a riga di comando saranno passati alla funzione tramite un array di puntatori.
e	L'insieme dei puntatori dell'environment è passato esplicitamente al processo figlio.
l	Gli argomenti a riga di comando sono passati individualmente alla funzione.

Esempio di utilizzo di `exec1`

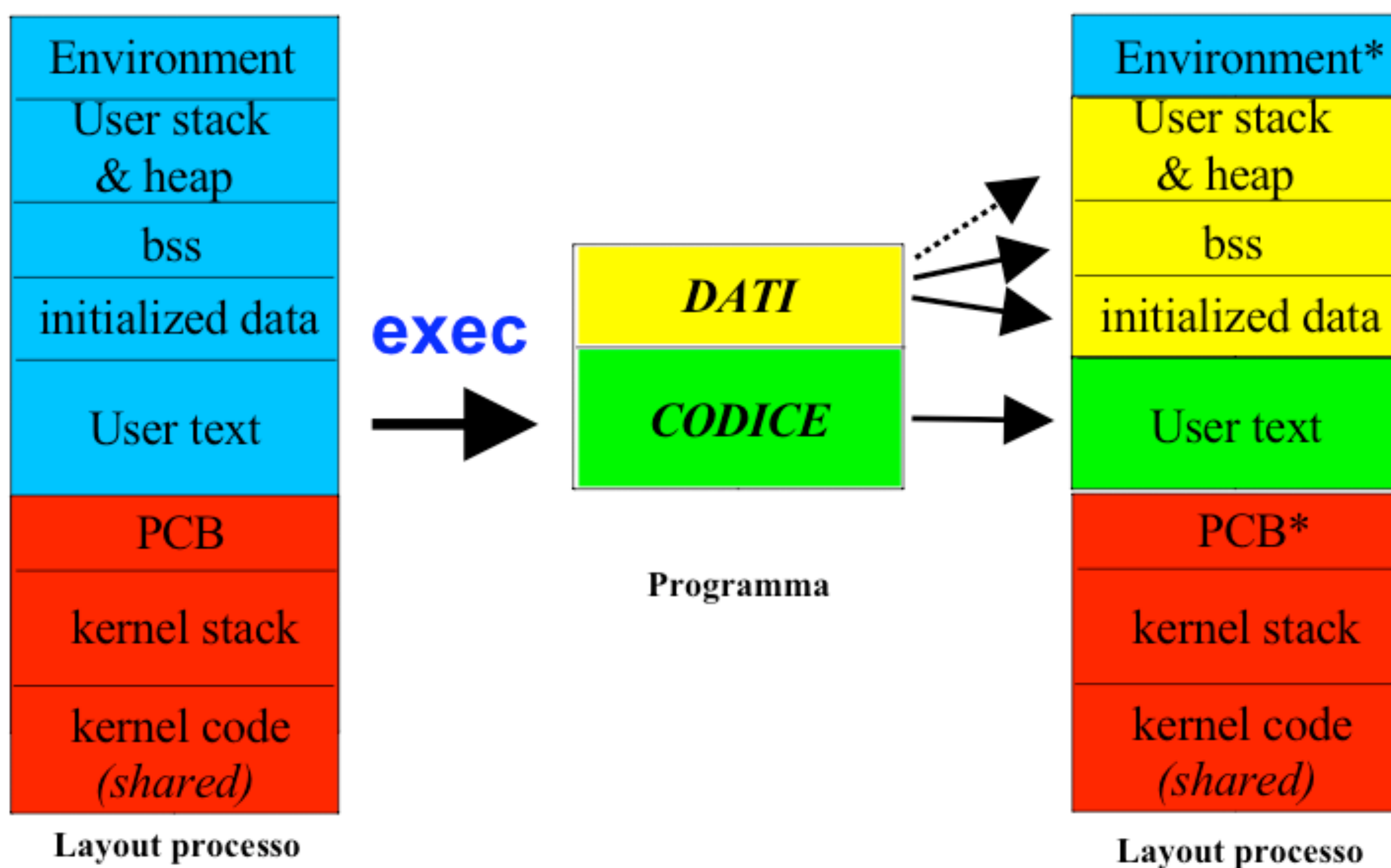
```
#include <stdio.h>
#include <unistd.h>
main()
{
    printf("Esecuzione di ls\n");
    exec1("/bin/ls", "ls", "-l", (char *)0);
    perror("La chiamata di exec1 ha generato un
    errore\n");
    exit(1);
}
```

- Si noti che `exec1` elimina il programma originale sovrascrivendolo con quello passato come parametro.
- Quindi le istruzioni che seguono una chiamata a `exec1` verranno eseguite soltanto in caso si verifichi un errore durante l'esecuzione di quest'ultima ed il controllo ritorni al chiamante.

Esempio di utilizzo di `exec1`



Esecuzione di Programmi



Chiamata alla system call `exec`

- Quando un processo chiama una delle system call `exec`:
 - il processo viene rimpiazzato completamente da un nuovo programma (text, data, heap, stack vengono sostituiti)
 - il nuovo programma inizia a partire dalla sua funzione main
 - il process ID non cambia
- Esistono sei versioni di `exec`:
 - con/senza gestione della variabile di ambiente `PATH`
 - se viene gestita la variabile d'ambiente, un comando corrispondente ad un singolo filename verrà cercato nel `PATH`
 - con variabili di ambiente ereditate / con variabili di ambiente specificate
 - con array di argomenti / con argomenti nella chiamata (null terminated)

Famiglia system call exec

```
int execl(char *pathname, char *arg0, ... );  
  
int execv(char *pathname, char *argv[]);  
int execlp(char *pathname, char *arg0, ..., char* envp[]);  
  
int execve(char *pathname, char *argv[], char* envp[]);  
  
int execlp(char *filename, char *arg0, ... );  
  
int execvp(char *filename, char *argv[]);
```

Eseguire Programmi

```
int execl(const char *pathname, const char *arg0, ...);
```

- execl accetta il nome di un programma da eseguire ed un numero variabile di argomenti per il programma
- l'ultimo argomento deve essere un puntatore nullo di tipo char*
- execl("a.out", "a.out", "xxx", (char *)NULL) esegue il programma a.out, con argomenti "a.out" e "xxx"

Eseguire Programmi

```
int execl(const char *pathname, const char *arg0, ...);
```

- se execl ha successo, il controllo non viene mai restituito al chiamante
 - il processo chiamante *diventa* il nuovo programma
- altrimenti, restituisce -1

Eseguire Programmi

- Per default, i file aperti dal processo corrente restano aperti dopo una exec
 - questo comportamento si può cambiare usando la system call `fcntl`
- Questo comportamento è utile per redirigere STDIN e STDOUT

Proprietà ereditate da `exec`

- Cosa viene ereditato da `exec`?
 - process ID e parent process ID
 - real uid e real gid
 - supplementary gid
 - process group ID
 - session ID
 - terminale di controllo
 - current working directory
 - root directory
 - maschera creazione file (umask)
 - file locks
 - maschera dei segnali
 - segnali in attesa

Eseguire Programmi

```
int execl(pathname, arg0, ...)  
int execv(pathname, char *const argv[])  
int execlp(filename, arg0, ...)  
int execvp(filename, char *const argv[])
```

Esempi d'uso:

```
char *args[] = {"ls", "-l", NULL};  
execvp("ls", args);
```

↕
equivalenti
↕

```
execlp("ls", "ls", "-l", NULL);
```

```
execl("ls", "ls", "-l", NULL);
```

errato: ls non si trova nella directory corrente

Famiglia Exec

```
# include <unistd.h>
```

```
int execl (const char *path, const char *arg0, ... /* (char *) 0 */);
```

```
int execvp (const char *file, const char *arg0, ... /* (char *) 0 */);
```

```
int execle (const char *path, const char *arg0, ... /* (char *) 0,  
                char *const envp[ ] */);
```

non ritornano se OK, ritornano -1 se EACCESS, EINVAL, EAGAIN, EINTR, ENOENT, ...

```
execl ("/home/gio/prog", "prog", "500", "out.txt", (char *) 0);
```

```
execvp ("prog", "prog", "500", "out.txt", (char *) 0);
```

```
char *env[ ] = {"USER=unknown", "PATH=/tmp", NULL};
```

```
...
```

```
execle ("/home/gio/prog", "prog", "500", "out.txt", NULL, env);
```

Famiglia Exec

```
# include <unistd.h>

int execv (const char *path, char *const arg[ ]);
int execvp (const char *file, char *const arg[ ]);
int execve (const char *path, char *const arg[ ], char *const
envp[ ]);
```

```
char *arg_vector[ ]= {"prog", "500", "out.txt", NULL};
```

```
...
```

```
execv ("/home/gio/prog", arg_vector);
```

```
execvp ("prog", arg_vector);
```

```
char *arg_vector[ ]= {"prog", "500", "out.txt", NULL};
```

```
char *env[ ] = {"USER=unknown", "PATH=/tmp", NULL};
```

```
...
```

```
execve ("/home/gio/prog", arg_vector, env);
```

Esercizio

- Trasformare il precedente programma utilizzando exec.

Utilizzo combinato di `fork` e `exec`

L'utilizzo combinato di `fork` per creare un nuovo processo e di `exec` per eseguire nel processo figlio un nuovo programma costituisce un potente strumento di programmazione in ambiente Linux

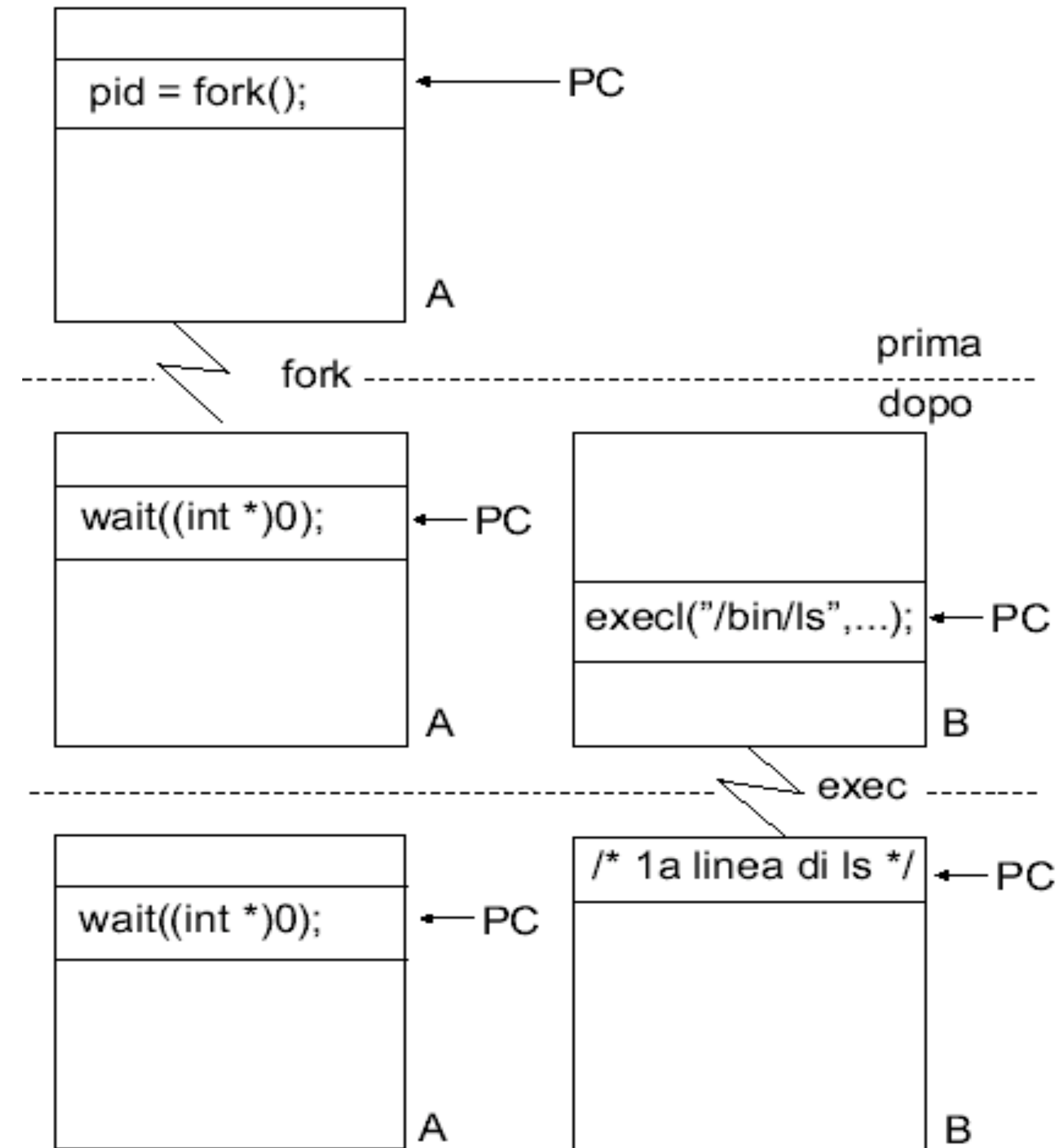
Esempio:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
void fatal(char *s)
{
    perror(s);
    exit(1);
}
/* continua ... */
```

Utilizzo combinato di `fork` e `exec`

```
main()
{
    pid_t pid;
    switch(pid = fork()) {
    case -1:
        fatal("fork failed");
        break;
    case 0:
        execl("/bin/ls", "ls", "-l", (char *)0);
        fatal("exec failed");
        break;
    default:
        wait((int *)0);
        printf("ls completed\n");
        exit(0);
    }
}
```

Utilizzo combinato di `fork` e `exec`



Esercizio 1

Scrivere un programma che manda in esecuzione un eseguibile il cui filename e' inserito come argomento sulla linea comando e ne aspetta la terminazione.

Aggiungere al programma precedente la capacità di lanciare eseguibili residenti in qualsiasi directory memorizzata nella variabile di ambiente `$PATH`.

Ambiente di un processo

- L'ambiente di un processo è un insieme di stringhe (terminate da \0).
- Un ambiente è rappresentato da un vettore di puntatori a caratteri terminato da un puntatore nullo.
- Ogni puntatore (che non sia quello nullo) punta ad una stringa della forma: identificatore = valore
- Per accedere all'ambiente da un programma C, è sufficiente aggiungere il parametro `envp` a quelli del main:

```
/* showmyenv */
#include <stdio.h>
main(int argc, char **argv, char **envp)
{
    while(*envp)
        printf("%s\n", *envp++);
}
```

- oppure usare la variabile globale seguente:

```
extern char **environ;
```

L'ambiente di un processo

- L'ambiente di default di un processo coincide con quello del processo padre.
- Per specificare un nuovo ambiente è necessario usare una delle due varianti seguenti della famiglia `exec`, memorizzando in `envp` l'ambiente desiderato:

```
execle(path, arg0, arg1, ..., argn, (char *)0, envp);  
execve(path, argv, envp);
```

L'ambiente di un processo

```
/* setmyenv */  
#include <unistd.h>  
#include <stdio.h>  
main()  
{ char *argv[2], *envp[3];  
  argv[0] = "setmyenv";  
  argv[1] = (char *)0;  
  envp[0] = "var1=valore1";  
  envp[1] = "var2=valore2";  
  envp[2] = (char *)0;  
  execve("./showmyenv", argv, envp);  
  perror("execve fallita");  
}
```

- ***Eseguendo il programma precedente si ottiene quanto segue:***

```
$ ./setmyenv  
var1=valore1  
var2=valore2
```

L'ambiente di un processo

- Esiste una funzione della libreria standard che consente di cercare in `environ` il valore corrispondente ad una specifica variabile d'ambiente:

```
#include <stdlib.h>
```

```
char *getenv(const char *name) ;
```

- `getenv` prende come argomento il nome della variabile da cercare e restituisce il puntatore al valore (ciò che sta a destra del simbolo =) o `NULL` se non lo trova:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
main()
```

```
{
```

```
    printf("PATH=%s\n", getenv("PATH")) ;
```

```
}
```

- Dualmente, `putenv` consente di modificare o estendere l'ambiente:
`putenv("variabile=valore") ;`

Current working directory e root directory

- Ad ogni processo è associata una current working directory che viene ereditata dal processo padre.
- La chiamata di sistema seguente consente di cambiarla:

```
#include <unistd.h>  
int chdir(const char *path);
```

- Ad ogni processo inoltre è associata una root directory che specifica il punto di inizio del file system visibile dal processo stesso.
- Per cambiare la root directory si può usare la chiamata di sistema seguente:

```
#include <unistd.h>  
int chroot(const char *path);
```

Esempio 7

```
/* exec.c: Dimostra il funzionamento di due funzioni exec */  
#include <sys/types.h> /* per il tipo pid_t */  
#include <unistd.h>    /* per le funzioni fork  execl ed execlp */  
#include <sys/wait.h>  /* per la funzione waitpid */  
#include <stdio.h>     /* per la funz. printf */  
#include <stdlib.h>    /* per la funzione exit */
```

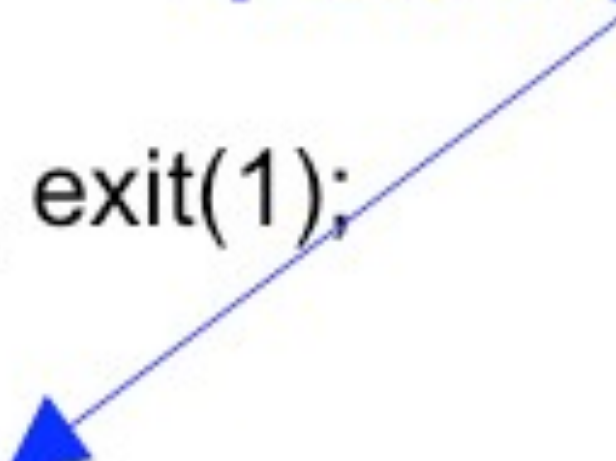
```
char *ambiente[ ] = { "USER = nobody", "PATH=/tmp", NULL };
```

```
int main(void)  
{  
    pid_t  pid;
```

```
    if ( (pid = fork()) < 0) printf("errore fork"), exit(1);
```

```
    else if (pid == 0) { /* 1mo figlio */  
        if (execl("/home/gio/echoall", "echoall", "1mo arg",  
            "2ndo e ultimo", NULL, ambiente) < 0)  
            printf("errore di execl"), _exit(1);
```

Esecuzione del programma echoall con
specificazione del pathname e dell'ambiente



Esempio 7 (cont.)

```
/* pid >0, genitore */
if ( (pid = waitpid(pid, NULL, 0)) < 0)
    printf("errore di waitpid"), exit(1);

if ( (pid = fork()) < 0)
    printf("errore di fork"), exit(1);

else if (pid == 0) {      /* 2ndo figlio */
    if (execvp("echoall", "echoall", "1mo e ultimo", NULL) < 0)
        printf("errore di execvp"), _exit(1);
}

/* pid >0, genitore */
if ( wait(NULL) != pid)
    printf("errore di wait"), exit(1);
exit(0);
}
```

Esecuzione del programma echoall senza
specificazione del pathname e dell'ambiente

```
/* echoall.c: Il programma eseguito da exec.c */

#include <stdlib.h>    /* per la funzione exit */
#include <stdio.h>     /* per la funzione printf */

void main(int argc, char *argv[ ])
{
    int          i;
    char          **ptr;
    extern char  **environ;

    for (i=0; i<argc; i++) /* effettua l'eco di tutti gli argomenti dalla
                               linea di comando... */
        printf("argv[%d] : %s\n", i , argv[i]);

    for (ptr=environ; *ptr != 0; ptr++) /* e di tutte le stringhe
                                           d'ambiente */
        printf("%s\n", *ptr);

    exit(0);
}
```

Esempio 7 (cont.)

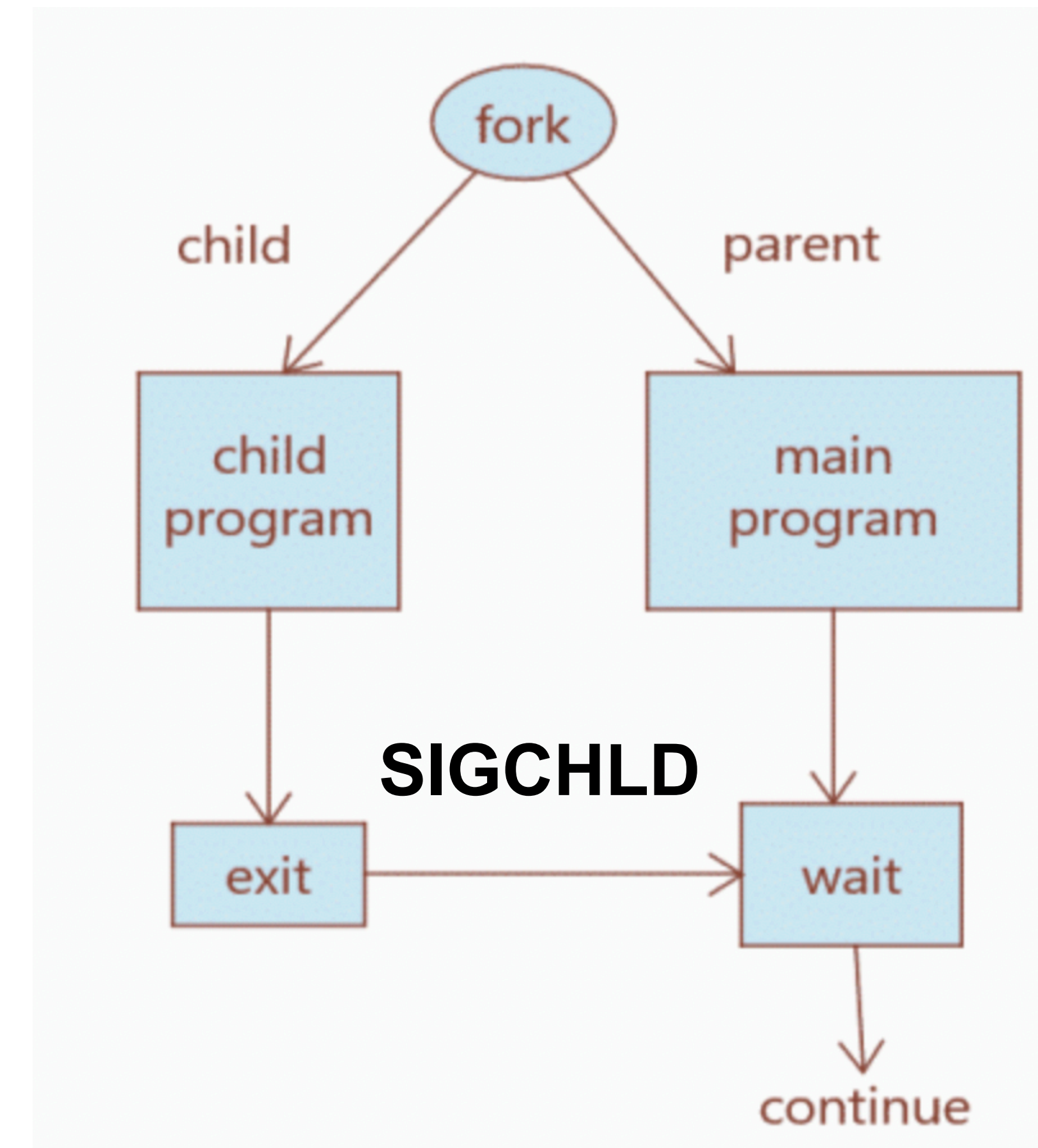
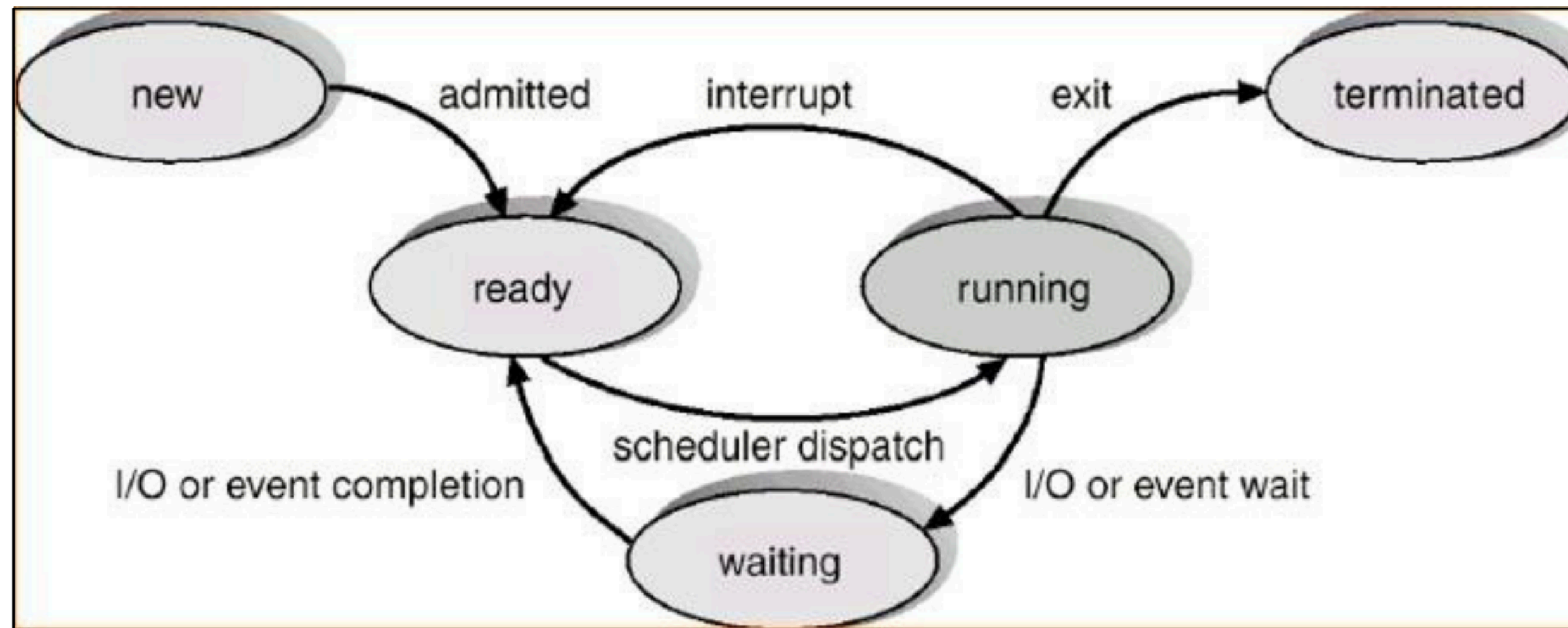
Compilando ed eseguendo, si ottiene:

```
gio$ pwd
/home/gio
gio$ cc -o echoall echoall.c
gio$ cc -o exec exec.c
gio$ echo $PATH
/usr/bin:/usr/local/bin:/home/gio
gio$ ./exec
argv[0] : echoall
argv[1] : 1mo arg
argv[2] : secondo e ultimo
USER=nobody
PATH=/tmp
argv[0] : echoall
argv[1] : 1mo e ultimo
USER=gio
HOME=/home/gio
...
```

Quando un processo termina

- viene inviato il segnale **SIGCHLD** al padre
- il processo diventa uno “zombie” finchè il padre non chiama una **wait/waitpid**

Ciclo di vita del processo



Terminazione Processo

- **Esistono tre modi per terminare in modo NORMALE:**
 - eseguire un return da main (è equivalente a chiamare `exit`)
 - chiamare la funzione `exit`:
 - `void exit(int status);`
 - invoca di tutti gli exit handlers che sono stati registrati
 - chiude di tutti gli I/O stream standard
 - è specificata in ANSI C
 - chiamare la system call `_exit`:
 - `void _exit(int status);`
 - ritorna al kernel immediatamente
 - è chiamata come ultima operazione da `exit`
 - è specificata nello standard POSIX.1

Terminazione Processo

- **Esistono due modi per terminare in modo ANORMALE:**
 - Quando un processo riceve certi segnali
 - generati dal processo stesso
 - generati da altri processi
 - generati dal kernel
 - Chiamando **abort**:
 - **void abort();**
 - La chiamata ad **abort** costituisce un caso speciale del primo caso dei tre sopra elencati, in quanto genera il segnale **SIGABRT**

NOTA: per informazioni sui segnali usa **man 7 signal**

Processi zombie

- **Cosa succede se il padre termina prima del figlio?**
 - il processo figlio viene "adottato" dal processo `init` (PID=1), in quanto il kernel vuole evitare che un processo divenga "orfano" (cioè senza un PPID)
 - quando un processo termina, il kernel esamina la tabella dei processi per vedere se aveva figli; in tal caso, il PPID di ogni figlio viene posto uguale a 1
- **Cosa succede se il figlio termina prima del padre?**
 - generalmente il padre aspetta mediante la funzione `wait` che il figlio finisca ed ottiene le varie informazioni sull'*exit status*
 - se il figlio termina senza che il padre lo "aspetti", il padre non avrebbe più modo di ottenere informazioni sull'*exit status* del figlio
 - per questo motivo, alcune informazioni sul figlio vengono mantenute in memoria e il processo diventa uno *zombie*

Aspettare un figlio

```
pid_t wait(int *status);
```

- Blocca il processo finché un figlio termina
 - non blocca se c'è un figlio zombie
- Restituisce il pid del processo terminato
 - -1 in caso di errore
 - ad esempio, se un processo non ha figli
- “status” contiene il valore di uscita del figlio
 - se non ci interessa, passiamo NULL

System call `wait` e `waitpid`

```
pid_t wait(int *status);
```

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- La `wait` e `waitpid` sono utilizzate per ottenere informazioni sulla terminazione dei processi figli
- Quando un processo chiama `wait` o `waitpid`:
 - può bloccarsi, se tutti i suoi figli sono ancora in esecuzione
 - può ritornare immediatamente con il termination status di un figlio, se un figlio ha terminato ed il suo termination status è in attesa di essere raccolto
 - può ritornare immediatamente con un errore, se il processo non ha alcun figlio
- Nota:
 - se eseguiamo una system call `wait` quando abbiamo già ricevuto `SIGCHLD`, essa termina immediatamente, altrimenti si blocca

System call `wait` e `waitpid`

- Significato degli argomenti:
 - `status` è un puntatore ad un intero; se diverso da `NULL`, il termination status viene messo in questa locazione
 - Il valore di ritorno è il process id del figlio che ha terminato
- Differenza tra `wait` e `waitpid`:
 - `wait` blocca il chiamante fino a quando un qualsiasi figlio non sia terminato
 - `waitpid` ha delle opzioni per evitare di bloccarsi
 - `waitpid` può mettersi in attesa di uno specifico processo
- Il contenuto del termination status dipende dall'implementazione:
 - bit per la terminazione normale, bit per l'exit status, etc.

Status wait waitpid

Se *statloc* non è NULL, l'intero cui esso punta rappresenta lo stato di terminazione del processo atteso, secondo una codifica che dipende dall'implementazione.

POSIX prevede che tale stato possa essere rilevato grazie ad opportune macro definite in `<sys/wait.h>`:

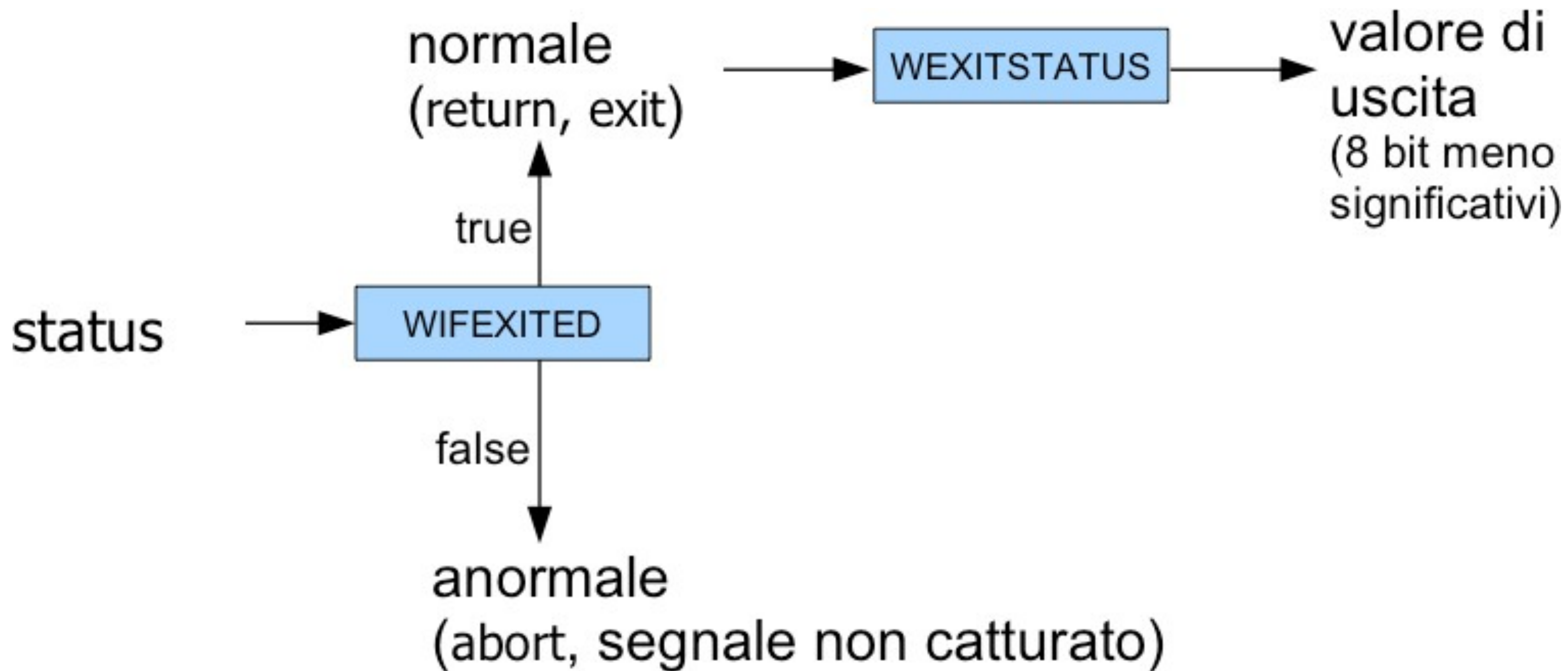
Macro	Descrizione
<code>WIFEXITED(status)</code>	vera se il figlio è terminato normalmente; in tal caso <code>WEXITSTATUS(status)</code> restituisce lo stato di uscita
<code>WIFSIGNALED(status)</code>	vera se il figlio è terminato a causa di un segnale; in tal caso <code>WTERMSIG(status)</code> ne restituisce il numero
<code>WIFSTOPPED(status)</code>	vera se il figlio è in stato di stop; allora il numero del segnale di stop è dato da <code>WSTOPSIG(status)</code>
<code>WIFCONTINUED(status)</code>	vera se il figlio ha ripreso l'elaborazione dopo uno stato di stop

Stato di uscita del Figlio

Esempio tipico:

```
int status;  
  
wait( &status);  
if ( WIFEXITED(status) )  
    printf("valore di uscita: %d\n", WEXITSTATUS(status));  
else  
    printf("terminazione anomala\n");
```

Stato del figlio



Aspettare un figlio

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- Come wait, ma aspetta un figlio specifico
- options può essere lasciato a zero

System call waitpid

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- Argomento **pid**:
 - **pid == -1** si comporta come **wait**
 - **pid > 0** attende la terminazione del figlio con process id uguale a **pid**
 - **pid == 0** attende la terminazione di qualsiasi figlio con process group ID uguale a quello del chiamante
 - **pid < -1** attende la terminazione di qualsiasi figlio con process group ID uguale a **-pid**
- Opzioni (parametro **options**):
 - WNOHANG** non si blocca se il child non ha terminato

Opzioni waitpid

waitpid dispone dell'argomento di tipo intero *options*, il cui valore può essere *zero* o una combinazione **OR bit a bit** (operatore `|`) con delle *costanti*, dipendenti dall'implementazione:

- se *options* = 0, **waitpid** attende per la terminazione di (almeno) un figlio specificato da pid;
- le costanti supportate da **POSIX** sono riassunte nella seguente tabella

Macro	Descrizione
WCONTINUED	ritorna lo stato di ogni figlio specificato da <i>pid</i> che ha ripreso l'elaborazione dopo uno stop
WNOHANG	waitpid ritorna immediatamente (con valore 0) se <i>pid</i> non individua alcun figlio che è terminato
WUNTRACED	ritorna lo stato di ogni figlio specificato da <i>pid</i> che è in stop ed il cui attuale stato non è stato riportato

La funzione `system`

```
int system(char* command) ;
```

- Esegue un comando, aspettando la sua terminazione
- E' una funzione della libreria standard definita in ANSI C (quindi non è una system call, anche se svolge una funzione analoga alla `fork+exec`)
- Il suo modo di operare è fortemente dipendente dal sistema; in genere chiama `/bin/sh -c command`
- Non è definita in POSIX.1, perché non è un'interfaccia al sistema operativo, ma è definita in POSIX.2

Funzione system

E' comodo poter eseguire un comando UNIX da un programma C. Per tale ragione l'ANSI C definisce la funzione `system`, che rappresenta un'interfaccia alla Bourne shell implementata mediante le primitive `fork`, `exec` e `waitpid`:

```
#include<stdlib.h>
int system(const char *cmdstring);
```

```
...
system("date >miofile");
...
```

```

/*
 * A simple implementation of system() (without signal handling)
 * Written by W. Richard Stevens
 */

#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>
#include <unistd.h>

int system(const char *cmdstring) /* version without signal handling */
{
    pid_t  pid;
    int status;

    if (cmdstring == NULL)
        return(1); /* always a command processor with Unix */

    if ( (pid = fork()) < 0) {
        status = -1; /* probably out of processes */

    } else if (pid == 0) { /* child */
        execl("/bin/sh", "sh", "-c", cmdstring, (char *) 0);
        _exit(127); /* execl error */
    } else { /* parent */
        while (waitpid(pid, &status, 0) < 0)
            if (errno != EINTR) {
                status = -1; /* error other than EINTR from waitpid() */
                break;
            }
    }

    return(status);
}

```



Università degli Studi di Napoli Federico II

THANK YOU FOR YOUR ATTENTION

TRUST ME ...
I AM A ROBOT!

